Universidad CENFOTEC

Escuela de Ingeniería del Software

SOFT-01 - Principios de Programación 1

TAREA 2

Inventario de una tienda

Estudiante: Daniel Barboza

Docente: Verónica Mora Lezcano

Fecha de entrega: 25 de octubre de 2026

Repositorio: https://github.com/dan13luchuz/tarea-2-principios-programacion

Rama: main

1. Enunciado del problema

Problema seleccionado: Inventario de una tienda. Una tienda desea registrar el inventario de sus productos. El sistema permite ingresar productos hasta que el usuario decida terminar.

- El ciclo se ejecuta mientras el usuario ingrese "s".
- Si el precio es menor o igual a cero, se muestra un error y el producto no se registra.
- Si la cantidad es menor a 5, el producto se marca como "requiere reposición".
- Se acumula el valor total del inventario (precio x cantidad).
- Al finalizar, se muestra el valor total, la cantidad de productos registrados y la cantidad que requiere reposición.

2. Datos de entrada y salida

Tipo	Dato	Tipo de dato	Uso
Entrada	nombre_producto	Texto	Nombre del producto.
Entrada	precio_unitario	Decimal	Precio de cada unidad.
Entrada	cantidad	Entero	Unidades en existencia.
Entrada	continuar	Texto	Controla la repetición: s / n.
Salida	Resumen final	Texto / números	Valor total, registrados y productos en reposición.

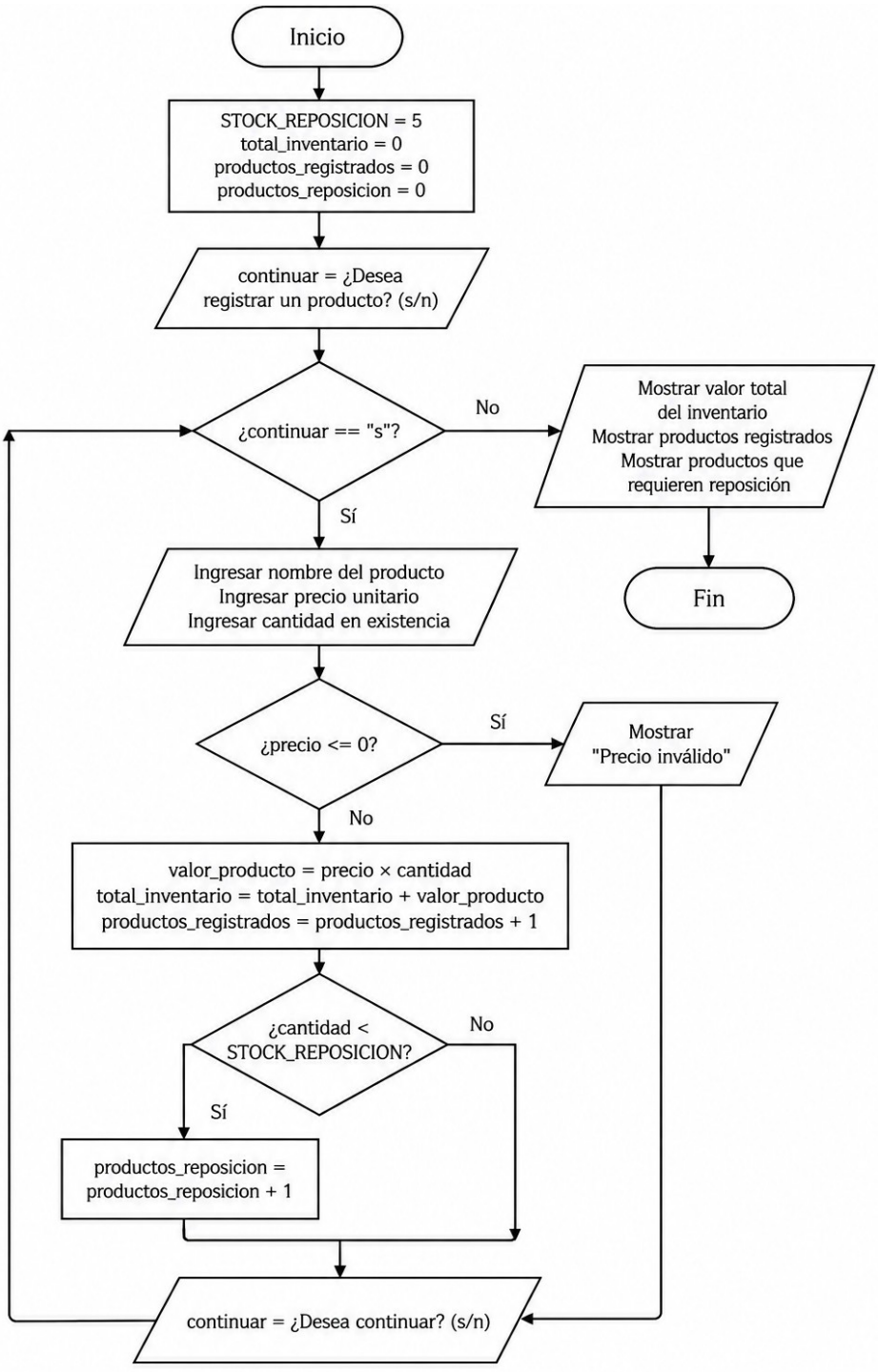
Constante: STOCK_REPOSICION = 5.

3. Algoritmo

1. Inicializar STOCK_REPOSICION = 5, el acumulador total_inventario y los contadores productos_registrados y productos_reposicion.
2. Preguntar si el usuario desea registrar un producto.
3. Mientras la respuesta sea "s", solicitar nombre, precio unitario y cantidad.
4. Si el precio es menor o igual a 0, mostrar un error. En caso contrario, calcular el valor del producto y actualizar el total y los contadores.
5. Si la cantidad es menor a 5, aumentar el contador de productos que requieren reposición.
6. Preguntar si desea continuar. Si responde "n", finalizar el ciclo.
7. Mostrar el resumen final del inventario.

4. Diagrama de flujo

El diagrama representa la misma secuencia utilizada en el programa Python, incluido el ciclo while.



5. Prueba de escritorio (traza)

Datos utilizados: Mouse (₡5 000, cantidad 8) y Cable (₡2 500, cantidad 4). Después del segundo producto, continuar = "n".

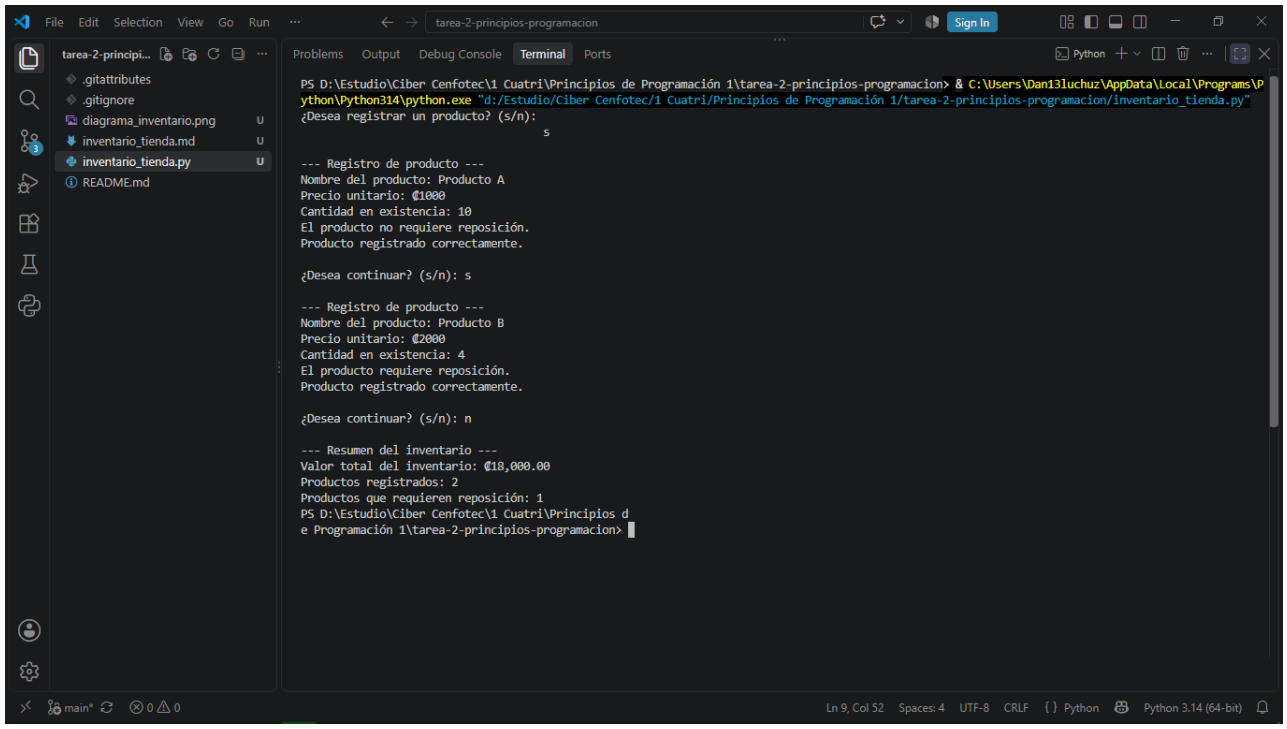
Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
0 (inicial)	continuar = "s"	-	-	Inicialización: total = 0; registrados = 0; reposición = 0.	Total = ₡0 Registrados = 0 Reposición = 0	-
1	continuar = "s"	"s" == "s" -> Verdadero	Sí	Mouse: 5000 x 8 = 40000. Total: 0 -> 40000. Registrados: 0 -> 1. 8 < 5 -> Falso. Reposición: 0 -> 0. continuar: "s" -> "s".	Total = ₡40 000 Registrados = 1 Reposición = 0	No requiere reposición. Producto registrado.
2	continuar = "s"	"s" == "s" -> Verdadero	Sí	Cable: 2500 x 4 = 10000. Total: 40000 -> 50000. Registrados: 1 -> 2. 4 < 5 -> Verdadero. Reposición: 0 -> 1. continuar: "s" -> "n".	Total = ₡50 000 Registrados = 2 Reposición = 1	Requiere reposición. Producto registrado.
Final	continuar = "n"	"n" == "s" -> Falso	No	No se ejecuta nuevamente el cuerpo del ciclo.	Total = ₡50 000 Registrados = 2 Reposición = 1	Valor total: ₡50 000. Registrados: 2. Reposición: 1.

Condición de finalización: el ciclo termina cuando continuar cambia a "n", por lo que la condición while continuar.lower() == "s" resulta falsa.

6. Pruebas del programa

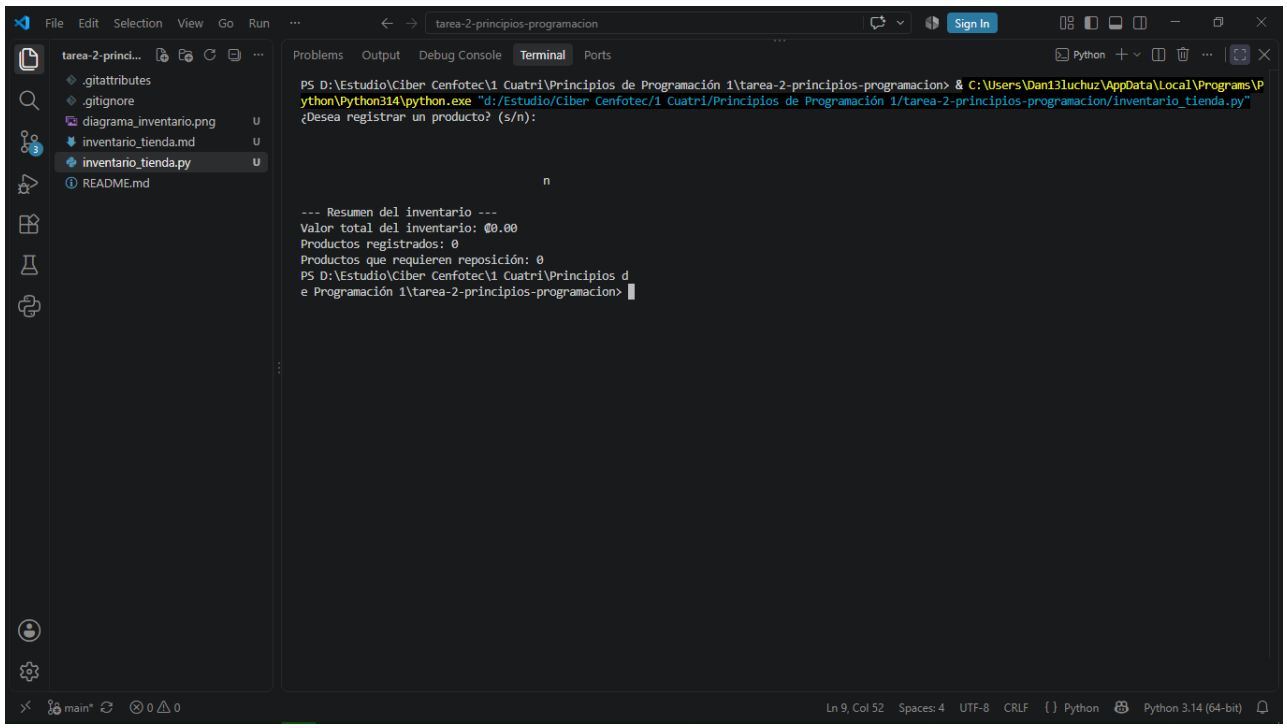
Se realizaron los cuatro casos requeridos: varias iteraciones, cero iteraciones, una sola iteración y caso límite.

Prueba 1 - Caso normal: varias iteraciones



Resultado: total ₡18 000.00; 2 productos registrados; 1 requiere reposición.

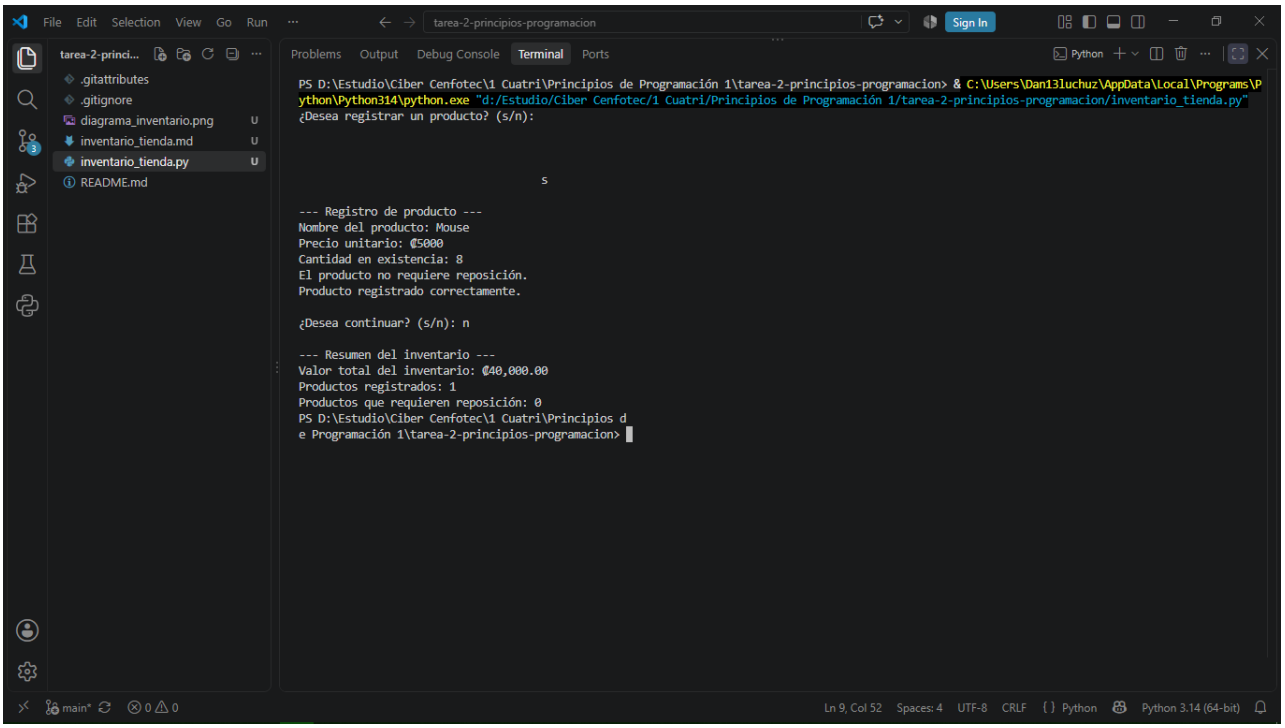
Prueba 2 - Cero iteraciones



Resultado: total ₡0.00; 0 productos registrados; 0 requieren reposición.

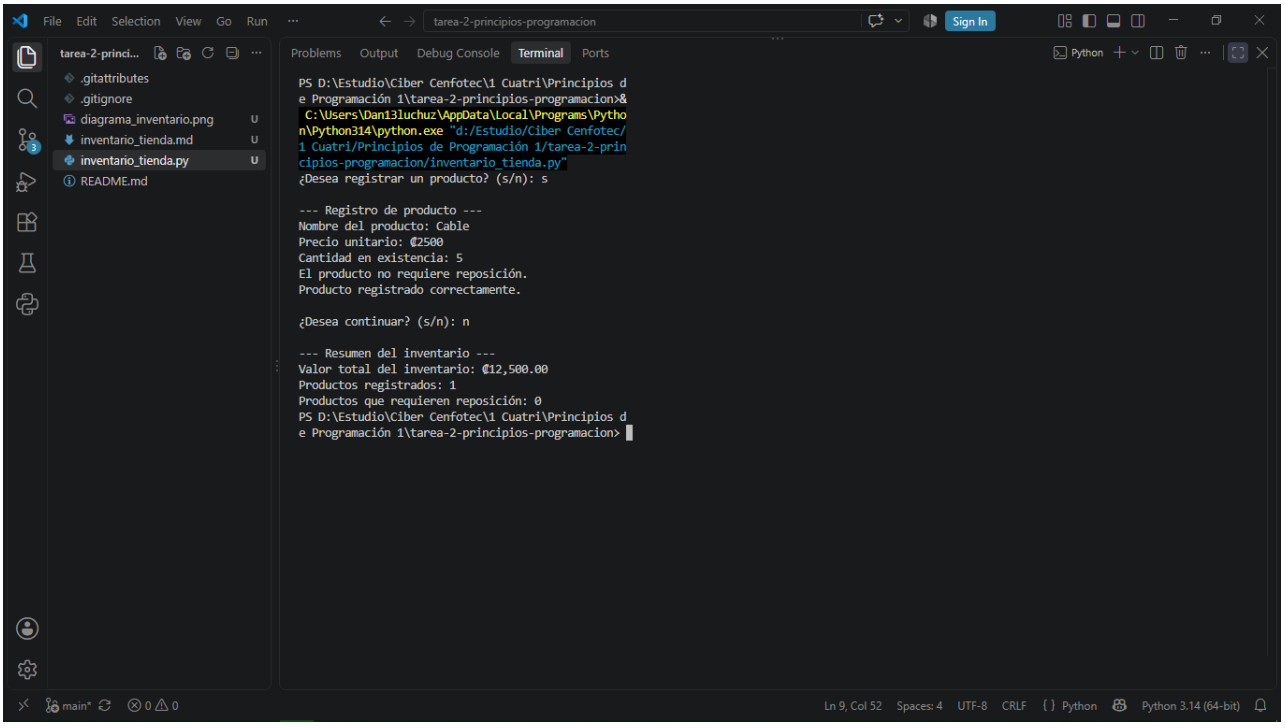
6. Pruebas del programa (continuación)

Prueba 3 - Una sola iteración



Resultado: total ₡40 000.00; 1 producto registrado; 0 requieren reposición.

Prueba 4 - Caso límite



Resultado: con cantidad = 5, el producto no requiere reposición. Total ₡12 500.00; 1 producto registrado; 0 requieren reposición.